

Санкт-Петербургский государственный университет
Факультет прикладной математики – процессов управления
ООП СВ.5005.2022 Прикладная математика, фундаментальная информатика и программирование

Шаго Павел Евгеньевич

*Математическое моделирование и реализация планировщика
процессов для гетерогенных процессорных архитектур*

Выпускная квалификационная работа

Научный руководитель:
кандидат физ.-мат. наук, доцент
Корхов В. В.

Рецензент:
эксперт по архитектуре и сетевому стеку,
ООО НИЦ АЭРОДИСК, Анохин Ю. В.

Санкт-Петербург
2026

Цель работы

разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению со штатным планировщиком ядра Linux

Предмет исследования

методы планирования, учитывающие различие вычислительной мощности ядер

Гетерогенные процессоры в современных устройствах

Сегмент	Примеры	Массово	Сегодня
Смартфоны	Apple A, Snapdragon, Dimensity, Exynos, Tensor	2013	все актуальные
Ноутбуки	Apple M, Intel Core Ultra, Ryzen AI, Snapdragon X	2020	все актуальные
Настольные ПК	Intel Core 12+, Intel Core Ultra	2021	частично
Серверы	NVIDIA GB10	—	экспериментально

Тренд: гетерогенные P/E-архитектуры, возникшие в мобильных SoC, стали нормой для процессоров ПК и ноутбуков. Серверные CPU пока остаются однородными

- 1 Проанализировать и определить наиболее перспективный подход к реализации планировщика
- 2 Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора
- 3 Реализовать разработанный алгоритм и собрать воспроизводимый набор бенчмарков
- 4 Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым планировщиком Linux EEVDF

Сравнение подходов к реализации планировщика

Критерий	В ядре	Модуль	gh0St	sched_ext
Без пересборки ядра	—	+	±	+
Устойчивость к ошибкам реализации	—	—	+	+
Низкие накладные расходы	+	+	—	±
Поддерживаемый интерфейс замены	—	—	±	+
Переносимость между версиями ядра	—	—	±	+

Вывод: sched_ext выигрывает по четырём критериям из пяти. Умеренные накладные расходы фреймворка остаются сознательным компромиссом за изоляцию ошибок и поддерживаемый интерфейс

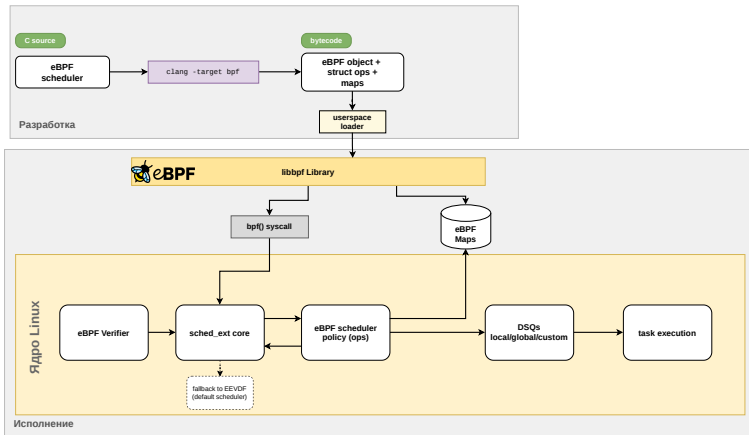


Рис. 1. Механизм работы sched_ext

DSQ (dispatch queue)

- SCX_DSQ_LOCAL
- SCX_DSQ_GLOBAL
- пользовательские DSQ

Цикл планирования

- select_cpu
- enqueue
- dispatch
- running / stopping

Аппаратная платформа

- Ядра: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$,
 $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$
- Дискретное время $t = 1, 2, 3, \dots$
- Замедление на E-ядре: $\sigma = \eta_P / \eta_E > 1$
- Общий ISA (любая задача может быть выполнена на любом ядре)

Задачи как агенты

- Тип $\theta_i = (v_i, l_i)$: ценность v_i , длина l_i
- Полезность как маргинальный вклад
 $U_i = W(s^t) - W_{-i}(s^t) \geq 0$
- Бюджет $B_i(\chi_i) > 0$ по классу обслуживания χ_i :
реального времени
интерактивный
пакетный
фоновый

Временные кванты P/E-ядер процессора – **недолговечный ресурс** (неиспользованный квант теряется безвозвратно). Их распределение моделируется динамическим аукционом VCG с задачами-агентами типа $\theta_i = (v_i, l_i)$, максимизирующим социальное благосостояние

Состояние $s^t = (\mathbf{x}^t, \mathbf{b}^t, \Theta^t, \mathcal{N}^t)$

- $\mathbf{x}^t$ – остаточные длительности контрактов
- $\mathbf{b}^t$ – остаточные бюджеты
- Θ^t – типы задач, $\mathcal{N}^t$ – активные задачи

Действие $a_i^t = (a_{i,P}, a_{i,E}) \in \{0, 1\}^2$,
 $a_{i,P} + a_{i,E} \leq 1$

$$\sum_i a_{i,P} \leq K_P^t, \quad \sum_i a_{i,E} \leq K_E^t$$

(K_κ^t – число свободных ядер кластера κ в момент t)

Эффективная ценность ($\kappa \in \{P, E\}$)

$$\phi_\kappa(\theta_i) = v_i - c_\kappa m_\kappa(l_i)$$

$$m_P(l) = l, \quad m_E(l) = \lceil l\sigma \rceil$$

Социальное благосостояние (Беллман, $\delta \in (0, 1)$):

$$W(s^t) = \max_{a^t} \left[\sum_i \phi_i^a + \delta \mathbb{E}[W(s^{t+1})] \right]$$

$$\phi_i^a = a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)$$

Платёж VCG

$$p_i^t = W_{-i}(s^t) - (W(s^t) - \phi_\kappa(\theta_i))$$

Решение аукциона. Задача направляется в одну из трёх кластерных DSQ по

$$\kappa^* = \arg \max_{\kappa} \phi_{\kappa}(\theta_i)$$

DSQ_P

$$\phi_P \geq \phi_E$$

P-ядро выгоднее по
эффективной ценности

DSQ_E

$\phi_E > \phi_P$ либо
короткая задача при
перегрузке P-кластера

$$Q_P \geq K_P \text{ и} \\ Q_P K_E > Q_E K_P$$

DSQ_Starved

$$B_i^t < B_i^{\max}/10$$

Низший приоритет,
обслуживается
после пополнения

Ослабления модели в реализации

- v_i наблюдается через вес задачи $p \rightarrow \text{scx.weight}$ (nice, cgroup)
- $W_{-i}(s^t) \rightarrow$ кластерная EWMA $\bar{W}_{\kappa}$ по реализованным $|\phi|$
- $l_i \rightarrow$ EWMA по истории исполнения
- Контракт – горизонт планирования, не атомарная единица
- Бюджет – корзина токенов с пополнением во сне ($B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$, $\Delta B_i \propto w_i$)

Система

Ядро Linux 7.0,
Arch Linux
CPU: Intel Core
Ultra 7 265K
8P + 12E, без HT

Сравнение

Linux EEVDF
(эталон)
scx_EEVDF
scx_LAVD
scx_A1349 (VCG)

Бенчмарки

- **hackbench** – группы процессов, обмен короткими сообщениями. Массовое порождение короткоживущих задач, стресс пробуждений и миграций
- **sysbench** – oltp_read_only, транзакции чтения к PostgreSQL. Потoki блокируются на ожидании ответа БД
- **schbench** – задержки планирования, веб-нагрузка. Запрос – две фазы сна и фаза матричных вычислений

Метрики

hackbench: время (с)
sysbench: TPS
schbench: RPS, p99 / p99.9 (мкс)
RAPL package: энергия (Дж)

Прогоны

3 уровня нагрузки
 $N = 6$, 95% ДИ

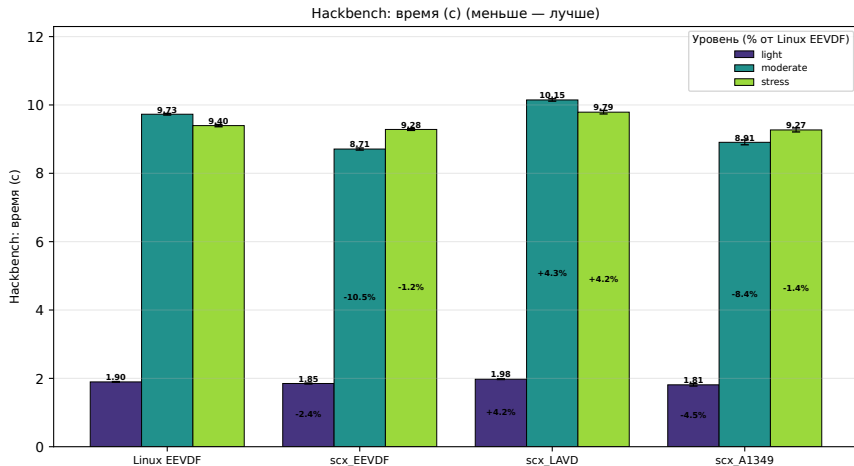


Рис. 2. Hackbench: время выполнения (с)

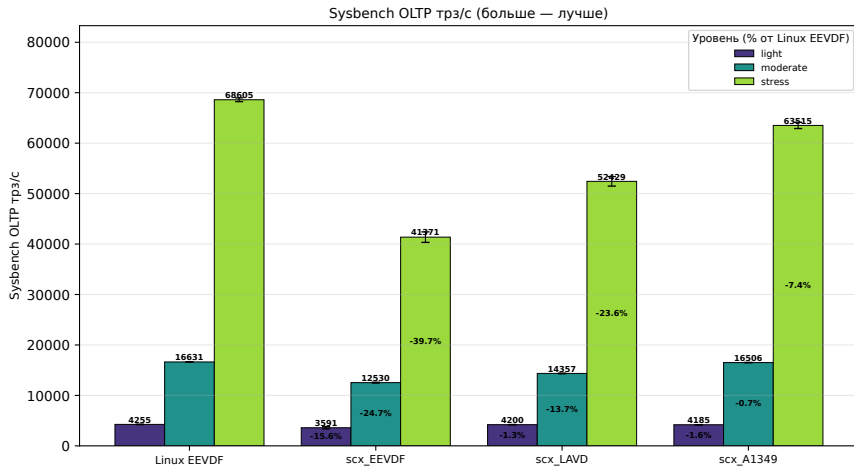


Рис. 3. Sysbench oltp_read_only: TPS

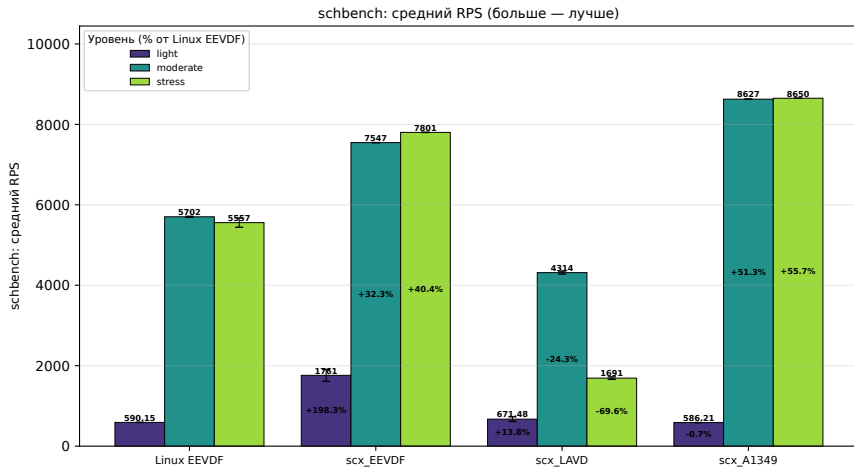


Рис. 4. Schbench: RPS

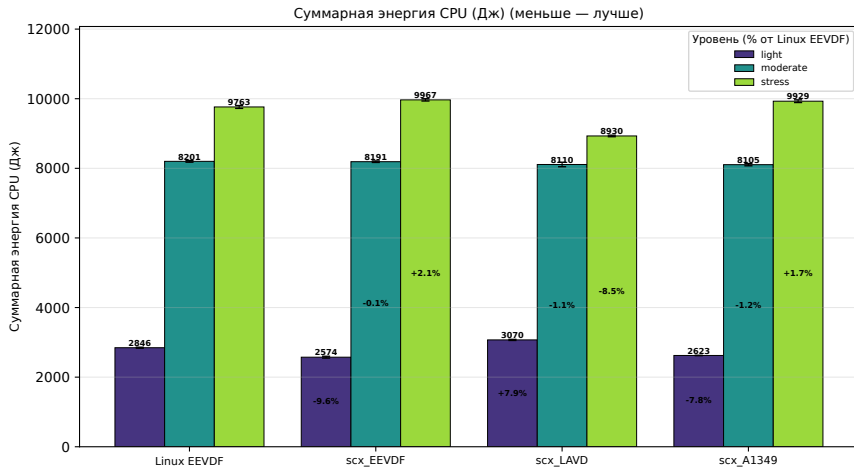


Рис. 5. Энергопотребление CPU за тест (Дж)

- На Intel Core Ultra 7 265K (8P + 12E) планировщик scx_A1349 превосходит штатный EEVDF на hackbench при трёх уровнях нагрузки на 4,5 / 8,5 / 1,4 %, а также лидирует по средней пропускной способности schbench при умеренной и стресс-нагрузке на 51,3 / 55,7 %. При стресс-нагрузке задержка обслуживания запроса p99.9 составляет 24,5 мс против $\approx 1,7$ с у штатного EEVDF
- Определена область неприменимости. На sysbench scx_A1349 уступает штатному EEVDF на 1,6 / 0,8 / 7,4 %. Все sched_ext-планировщики отстают, причём scx_EEVDF без учёта гетерогенности – сильнее (до 39,7 %), что указывает на накладные расходы фреймворка при высоком темпе пробуждений на IPC
- Атрибуция эффектов конкретным факторам реализации (аукционный платёж, корзина токенов, выбор P-ядра, привязанные к ядрам очереди) остаётся гипотетической, разделение вкладов требует серии экспериментов с поочерёдным отключением факторов

- Проведён сравнительный анализ подходов к реализации планировщика и выбран sched_ext
- Построена математическая модель распределения квантов P/E ядер как динамического аукциона VCG
- Реализован планировщик scx_A1349 на основе построенной модели
- Проведён эксперимент на реальном гетерогенном оборудовании, проведено сравнение со штатным планировщиком Linux и современными eBPF-планировщиками

Спасибо за внимание!

- EEVDF** Earliest Eligible Virtual Deadline First — штатный планировщик ядра Linux с 6.6
- sched_ext** инфраструктура ядра для подключаемых планировщиков на eBPF
 - eBPF** extended Berkeley Packet Filter, верифицируемый байт-код, исполняемый в ядре
 - ghOSt** пользовательский планировщик от Google, политика в user-space
 - scx_LAVD** Latency-Aware Virtual Deadline, eBPF-планировщик с учётом отзывчивости
 - scx_EEVDF** порт EEVDF поверх sched_ext, эталон накладных расходов фреймворка
 - DSQ** Dispatch Queue, очередь готовых задач внутри sched_ext
 - VCG** Vickrey-Clarke-Groves, аукцион с правдивыми оценками
 - MDP** Markov Decision Process, марковский процесс принятия решений
 - EWMA** Exponentially Weighted Moving Average, экспоненциально-взвешенное скользящее среднее
 - RAPL** Running Average Power Limit, счётчики энергии Intel
- hackbench** бенчмарк IPC через сокеты/каналы, короткоживущие процессы
- sysbench oltp_read_only** OLTP-нагрузка чтения к PostgreSQL
 - schbench** бенчмарк хвостовых задержек планирования от Chris Mason
- p99 / p99.9** 99-й и 99,9-й перцентили задержки
- P / E ядра** Performance / Efficient ядра в гибридной архитектуре Intel
 - σ коэффициент замедления, отношение пиковой производительности P-ядра к E-ядру

Пример: $K_P = K_E = 1$, одно свободное ядро типа κ

Победитель:

$$i^* = \arg \max_{i \in \mathcal{N}^t, B_i^t > 0} \phi_\kappa(\theta_i) + \delta^{m_\kappa(l_i)} \bar{W}_\kappa$$

где $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$

Платёж победителя:

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa$$

где j — задача на втором месте, $\bar{W}_\kappa$ — ожидаемое будущее благосостояние кластера.

$\phi_P(\theta_i) > \phi_E(\theta_i)$ выполняется не всегда. При малом v_i и большом l_i E-ядро может дать большую эффективную ценность ($c_E \lceil l_i \sigma \rceil < c_P l_i$ при $\sigma < c_P / c_E$). Оптимальная стратегия нетривиальна.